

Miniteste M3 — Interfaceamento paralelo e display

Microcontroladores · DENE/UFMT · GABARITO

Nome: _____

20 minutos

Consulta permitida à folha de referência do HD44780 — **nenhuma questão se responde procurando**. As respostas valem pelo raciocínio; um número certo sem justificativa vale metade.

Questão 1 (2,0)

Um estudante escreve o código abaixo esperando ver a letra **A** na tela. O display foi inicializado corretamente antes.

```
LATE0 = 1;    /* RS */
LATE2 = 0;    /* R/W */
LATD = 0x41;
__delay_us(40);
```

- (a) O que aparece na tela? Justifique em uma frase.
- (b) Um colega sugere: “aumenta o atraso para 1 ms que resolve”. Avalie a sugestão.

RESPOSTA

- (a) Nada. O valor fica parado nos fios e o controlador o ignora: a linha **E** nunca subiu, e portanto nunca desceu. O nível prepara; é a borda que transfere.
- (b) Não resolve, e nenhum valor de atraso resolveria. O problema não é quantidade de tempo — é a ausência de um evento. O controlador não amostra o barramento continuamente; ele amostra na borda de descida de **E**. Sem essa borda, esperar mil vezes mais é esperar por nada.

Questão 2 (2,0)

Duas versões da mesma linha:

```
LATD = (LATD & 0x0F) | (valor & 0xF0);
LATD = (PORTD & 0x0F) | (valor & 0xF0);
```

- (a) Descreva concretamente o defeito da segunda versão, supondo que os quatro bits baixos do **PORTD** estejam ligados a LEDs.
- (b) O estudante desconecta os LEDs para investigar e o defeito desaparece. Explique por quê — e diga o que isso significa para quem procura o erro.

RESPOSTA

- (a) A operação é na verdade três: ler os oito bits, alterar quatro, escrever os oito de volta. Lendo de **PORTD**, o que se lê é o nível elétrico do pino, e não o que foi escrito nele. Um pino carregando um LED pode ainda não ter subido ao nível escrito no instante da leitura; lê-se 0 onde havia 1, e esse 0 é reescrito. O LED apaga sozinho, sem que nenhuma linha do programa tenha mandado apagá-lo.
- (b) Sem carga, o pino atinge o nível quase instantaneamente e a leitura passa a coincidir com o latch. O defeito não some: some o sintoma. E é a pior categoria de defeito justamente por isso — a ação de investigar é a ação que o esconde. Só o raciocínio sobre o mecanismo encontra este erro; tentativa e erro na bancada, não.

Questão 3 (2,0)

Um programa posiciona o cursor no endereço **0x00** e escreve 20 caracteres seguidos, sem nenhum outro comando.

- (a) Quantos caracteres aparecem, e onde estão os demais?
- (b) Sem reescrever nenhum caractere, é possível fazer o 17.^o aparecer na tela? Se sim, como.

RESPOSTA

(a) Aparecem 16, todos na primeira linha. Os quatro restantes foram gravados em **0x10** a **0x13** — posições que existem na memória do controlador, guardam o que foi escrito nelas, e não correspondem a nenhum ponto da tela. Não passam para a segunda linha: a segunda linha começa em **0x40**, e o contador de endereços não salta.

(b) Sim, com um comando de deslocamento (**0001 1x****). A tela é uma janela de 16 posições sobre 40; deslocar move a janela, não o conteúdo. Os caracteres continuam onde estão e passam a cair dentro da parte visível.

Aceitar também: reposicionar o cursor em **0x40** e reescrever — mas isso reescreve, e o enunciado proíbe.

Questão 4 (2,0)

O kit está ligado. Você aperta SW9, o microcontrolador reinicia e o display permanece energizado o tempo todo.

(a) Por que o controlador pode estar em **três** estados, e não em dois?

(b) Um estudante remove uma das três repetições de **0x30**. Descreva uma situação em que o programa continua funcionando e uma em que falha.

RESPOSTA

(a) Os dois estados óbvios são interface de 8 bits e interface de 4 bits. O terceiro existe porque, em 4 bits, um byte são dois pulsos — e o reset pode cair **entre** os dois. O controlador fica esperando a segunda metade de um byte cuja primeira metade veio do programa anterior.

(b) **Funciona** quando o display acabou de ser energizado junto com o microcontrolador: aí ele está seguramente em 8 bits, e o primeiro **0x30** já é o comando que se queria. **Falha** quando só o microcontrolador foi reiniciado e o controlador estava esperando a segunda metade de um byte: são necessários três passos para levar os três estados ao mesmo lugar, e com dois um deles ainda não chegou.

O detalhe que vale ponto: é justamente o reset por SW9 — o gesto mais comum do laboratório — que produz o estado que a sequência encurtada não resolve.

Questão 5 (2,0)

(a) No driver, a linha **LATE2 = 0** é escrita explicitamente antes de cada byte, embora **0** pareça ser o valor natural do pino. Por que essa linha não é redundante? Que tipo de sintoma sua ausência produz?

(b) Uma atualização completa de tela custa cerca de 1,3 ms. O gerador de melodia da bancada inverte um pino a cada 1,14 ms para produzir um lá. Escrever no display e tocar a nota no mesmo laço funciona? Justifique com os dois números.

RESPOSTA

(a) O pino sai do reset como **entrada**, não como saída em nível baixo. Sem escrever no latch e configurar o **TRIS**, **R/W** fica flutuando, e o módulo pode entender leitura onde o programa quis escrita. O sintoma é intermitente: funciona, funciona, funciona, e um dia não — sem que nada no código tenha mudado. Vale mencionar que é um defeito sem sintoma reproduzível, e por isso capaz de sobreviver meses num projeto.

(b) Não funciona. Uma atualização de tela é **mais longa** que meio período da nota, e durante ela o processador está bloqueado esperando o controlador. O pino não é invertido a tempo, e a nota sai errada toda vez que o display é escrito — um estalo, ou queda de tom. Nenhuma reorganização do laço resolve enquanto a escrita for bloqueante; a saída é o processador ser interrompido durante a espera.